Anti-Pattern: Caches Treated as Authoritative — Standalone Formalization of the Failure Mode Where Performance Optimization Stores Become Operationally Authoritative for Coordination Questions, Violating Substrate-as-Source-of-Truth and Pattern B's Non-Authoritativeness in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 6, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one specific failure mode of two foundational architectural commitments — substrate-as-source-of-truth and hybrid systems composition through the substrate-derived view pattern — as a standalone anti-pattern: deployments in which performance caches are treated as operationally authoritative for coordination questions, violating Pattern B's non-authoritativeness commitment specifically.

Abstract

The CKS pattern commits substrate to be the source of truth for coordination across five categories — what is the case, what is current, what is in conflict, what rules apply, and who has what authority — and commits the legitimate composition of CKS substrates with adjacent components to occur through three patterns, one of which (Pattern B: substrate-derived view) governs caches and projections as non-authoritative views generated from substrate state. When caching layers — Redis, Memcached, in-memory caches, CDN caches, query-result caches, embedding caches, computed-value caches, ORM-level caches, HTTP response caches — operate as authoritative for coordination questions rather than as performance accelerators, the architectural failure cuts through both commitments simultaneously: substrate-as-source-of-truth fails because cache content becomes the consulted source, and Pattern B fails because its third operational component (non-authoritativeness) ceases to hold. This note states the anti-pattern's four operational components, identifies the CKS commitments it violates, traces the downstream failure mode, specifies the architectural correction, distinguishes it from four adjacent legitimate patterns, and provides an operational test with three sharpening properties. The note closes a five-anti-pattern cluster covering source-of-truth migration to five distinct locations: agent infrastructure, LLM context, cell internals, external systems, and (here) performance optimization stores.

1. Why caches-as-authoritative needs to be formalized as standalone

The CKS pattern's foundational commitment that **substrate is the source of truth** (§11.3) names substrate as authoritative across five coordination categories. The foundational commitment to **hybrid systems composition** (§4.5) names three legitimate patterns by which CKS substrates compose with adjacent AI components: input-style (Pattern A), substrate-derived view (Pattern B), and separate concern (Pattern C). Pattern B has three operational components — substrate-derived, regeneratable from substrate, and non-authoritative — all three of which must hold for a derived view to be a legitimate compose-point rather than a substrate substitute.

Performance caches are architecturally Pattern B derived views. A cache stores content derived from substrate state (substrate-derived); the cache content can be regenerated from substrate state on cache miss or invalidation (regeneratable); the cache content is, by Pattern B's commitment, not the consulted source for coordination questions (non-authoritative). When the third component fails — when cache content begins serving authoritative answers — the architectural failure is not merely operational drift; it is two foundational CKS commitments breaking simultaneously through the same mechanism.

Motivating cases include CKS deployments placing Redis or another caching layer in front of substrate where cached query results are treated as authoritative; deployments with CDN-cached content authoritative for queries; application-level caches as the operational source consulted by downstream components; embedding caches authoritative for semantic relationship queries; computed-value caches authoritative for derived facts; cache-aside patterns in which cache hits never trigger substrate consultation; and ORM-level caches where the ORM's cached entity state is treated as authoritative.

Standalone formalization matters because caching is universal in 2024–2026 deployments and AI products commonly include extensive caching for embeddings, computed values, and query results. Patentable derivations focused on AI-driven cache architectures, embedding caching for AI systems, or “performance-optimized AI” architectures are substantially more contestable when the anti-pattern they would normalize is publicly formalized as derivative prior art. The note also closes the source-of-truth anti-pattern cluster — together with prior cluster notes (agent memory, LLM context, hidden cell state, external tool state), this note covers source-of-truth migration to performance optimization stores.

2. The anti-pattern, defined precisely

A deployment exhibits the **caches-as-authoritative** anti-pattern when all four of the following operational components hold.

(a) Cache holds coordination-scope content. The deployment's cache — at any layer — contains content in one or more of the five source-of-truth categories: what is the case, what is current, what is in conflict, what rules apply, who has what authority. The content carried by the cache architecturally falls within substrate's authoritative scope.

(b) Cache is consulted as authoritative for coordination questions. Coordination questions are routed to cache; cached content provides answers without those answers being validated against substrate. Operations downstream of the cache treat the returned content as the operational truth.

(c) Cache hits bypass substrate consultation. Operations that should consult substrate consult cache instead when cache hits are available. The pattern “cache hit → return cached content” treats cache as the primary read path; substrate consultation becomes the rare exception (cache miss) or never occurs at all where cache hits dominate.

(d) Cache staleness produces authoritative-looking answers diverging from substrate. Cached content may be stale relative to current substrate state; under the anti-pattern, the stale content is operationally authoritative even when it diverges. The architectural commitment to authoritative content currency fails through staleness becoming authoritative.

The four components together define the anti-pattern. A deployment exhibiting any one partially exhibits it; a deployment exhibiting all four exhibits it fully.

3. Which CKS commitments are violated

Substrate-as-source-of-truth (§11.3) — directly violated. The foundational commitment that substrate is authoritative across the five coordination categories fails when cache content provides authoritative answers instead.

Hybrid systems composition (§4.5) — directly violated. The foundational commitment to three legitimate composition patterns fails when Pattern B's operational boundary is breached.

Pattern B (substrate-derived view) — directly and uniquely violated through its third operational component. Pattern B's three operational components are substrate-derived, regeneratable-from-substrate, and non-authoritative. Caches satisfy the first two by construction. The anti-pattern is the failure of the third specifically: cache content becomes operationally authoritative, and Pattern B's commitment to non-authoritativeness ceases to hold. This violation is uniquely diagnostic — it distinguishes caches-as-authoritative from sibling source-of-truth anti-patterns at locations where Pattern B does not architecturally apply. It is also the specific cache-layer instantiation of the named composition anti-pattern "Pattern B derived views as substrate substitutes."

Categories 1 and 2 — directly violated when cache content is authoritative for "what is the case" and "what is current"; the latter is operationally severe because cache staleness specifically affects currency answers. **Categories 3, 4, 5** — potentially violated when cache content is authoritative for those categories.

Read determinism (§6.2) — directly violated. Reads of cached "authoritative" content may return content diverging from substrate state; the same coordination question may produce different authoritative answers depending on cache state, time of consultation, and which cache instance is consulted in distributed deployments. The determinism contract more broadly (§6.2, §11.3) is extended-implicated, since cache staleness, invalidation timing, and distributed cache divergence introduce non-determinism not in the categories the contract permits.

Three further commitments are extended-implicated. Path retraceability (§3.1) is implicated because cache hits may not record consultation against substrate; the retraceable trail may have gaps at cache-hit moments where the recorded rationale is "cache returned X" rather than "substrate state was Y when consulted." AI-as-substrate-mediator (§4.1, §4.2) is implicated when LLMs consult caches and treat cache content as authoritative; the mediator role assumes substrate as the read source, not cache. Human-governed (§2.1, §3.3) is implicated because humans exercising the inspect right inspect substrate; cache content operating as authoritative makes the inspect right operationally compromised because authority lives in cache layers humans do not typically govern through the substrate's inspection mechanisms — and per-substrate human governance preservation is similarly implicated when governance does not extend to caches but caches operationally exercise authority.

4. The failure mode

Caches-as-authoritative produce deployments where coordination authority depends on cache content that may diverge from substrate. The downstream consequences are operationally specific.

Cache staleness causes divergent authoritative answers. Operations consulting cache may receive answers that no longer match substrate; subsequent operations consulting substrate (on cache miss, or by direct query) may receive different answers. The deployment operates with mismatched authoritative content depending on which path was taken.

Cache invalidation failures cause persistent stale authority. Invalidation logic may not propagate across distributed caches, may fail under network partition, or may have bugs that miss specific update patterns. Persistent stale content becomes persistently authoritative.

Cache-warming and cache-aside both migrate authority away from substrate. Cache-warming patterns populate caches eagerly so subsequent operations consult cache as the primary read path; substrate consultation becomes the rare exception, and the architectural commitment to substrate as source of truth fails through operational read-path migration. Cache-aside patterns may have writes that update cache without updating substrate, or that update substrate without invalidating cache; in either case the architectural commitment fails through write-path divergence.

Embedding caches become authoritative for semantic queries. AI deployments commonly cache vector embeddings to avoid recomputation. Under the anti-pattern, the cached embeddings become authoritative for semantic relationship queries; if substrate content changes but the embedding cache is not refreshed, the deployment operates with stale semantic authority — semantically retrieving content that no longer exists or no longer matches.

Computed-value caches become authoritative for derived facts. Caches storing derived calculations may become authoritative for facts that should be recomputed from substrate. Pattern B's second operational component (regeneratable from substrate) is technically still satisfied; the third (non-authoritative) has failed. In distributed multi-instance deployments, caches across instances may further diverge — different operations consulting different cache instances see different “authoritative” content, and the deployment operates with instance-dependent authority.

The “performance optimization” framing masks architectural failure. Caching is universally considered good engineering; deployment teams may resist architectural correction because the anti-pattern is positioned as an operational virtue. Cache-induced consistency violations across the five categories arise when cache content for one category is stale while substrate is updated, and recovery is operationally constrained — it requires identifying which cached content is stale and invalidating it, which is difficult for large caches with complex invalidation patterns.

5. The architectural correction

The architectural correction operates through three foundational commitments together.

Substrate as single source of truth across the five categories. Substrate is authoritative for coordination questions; cache content is non-authoritative reference information used for performance optimization.

Pattern B preservation. Caches operate as Pattern B derived views with all three operational components — substrate-derived, regeneratable-from-substrate, and non-authoritative. The architectural commitment is

that caches accelerate substrate access without replacing substrate authority.

Substrate consultation for coordination questions. Operations needing answers to coordination questions consult substrate; cache may provide performance optimization, but the authoritative answer comes from substrate. Cache misses must hit substrate; cache hits must validate against substrate authority for coordination-critical operations.

A correctly architected deployment additionally:

- **Treats caches as performance acceleration, not authority.** The pattern is substrate-authoritative-with-cache-acceleration, not cache-authoritative-with-substrate-fallback.
- **Invalidates cache content on substrate change.** When substrate state changes, cache content for the changed state is invalidated. Persistent stale cache content is the failure mode; reflecting current substrate or invalidating is the commitment.
- **Validates cache content for coordination-critical operations.** Coordination-critical operations may bypass cache entirely. Cache acceleration is applied where staleness is acceptable; coordination authority requires substrate consultation regardless of cache state.
- **Distinguishes cache-aside from cache-as-authoritative.** Cache-aside patterns where writes update substrate first and then invalidate cache are legitimate when substrate remains authoritative. Cache-aside patterns where reads consult cache without substrate validation for coordination-critical operations exhibit the anti-pattern.
- **Maintains a cache-locus audit.** Tests verify that downstream operations consult substrate (not cache) for authoritative answers; cache provides performance optimization without replacing authority.
- **Reframes "performance optimization" within Pattern B constraints.** Performance optimization that violates substrate-derived, regeneratable, or non-authoritativeness exhibits the anti-pattern even when it improves measured latency.

6. What the anti-pattern is NOT

Four adjacent legitimate patterns are commonly conflated with caches-as-authoritative.

Not Pattern B derived views that respect non-authoritativeness. Derived views that are regeneratable from substrate and that remain non-authoritative for coordination questions satisfy the architecture. Caches that respect non-authoritativeness are legitimate Pattern B compose-points; the anti-pattern is the failure of the third operational component, not the existence of caches.

Not performance caches in general. Performance caches that exist to reduce substrate read latency, where coordination questions consult substrate and caches provide acceleration, are legitimate. The anti-pattern is the operational migration of authority into cache, not the practice of caching.

Not cache-aside patterns with substrate-authoritative writes. Cache-aside patterns where writes update substrate first, invalidate cache, and reads consult cache for performance with substrate-fallback for coordination-critical operations are legitimate. The anti-pattern is the cache-aside variant where writes do not invalidate cache or where reads do not validate against substrate for coordination authority.

Not read-through caches with substrate authority preserved. Read-through caches that load substrate content into cache on cache miss and serve cached content for subsequent reads are legitimate when

substrate remains authoritative for coordination. The anti-pattern arises when read-through traffic causes coordination authority to migrate to the cache layer in practice.

7. Why caches-as-authoritative is load-bearing as an anti-pattern

The anti-pattern is operationally common in 2024–2026 deployments — caching is universal in performance-conscious architectures and AI products commonly include caches for embeddings, computed values, and query results. It violates Pattern B's non-authoritativeness component specifically — the third operational component and the one most easily breached without architectural awareness. It introduces unique risks among source-of-truth anti-patterns: cache staleness, invalidation failures, distributed divergence, and cache-warming side effects are risks specific to performance optimization stores. It compounds with sibling anti-patterns at LLM, agent-memory, cell-internal, and external-tool locations. It is detectable through architectural review using the operational test in §8. Its architectural correction is specific and tractable. And it closes the source-of-truth anti-pattern cluster: with this note, source-of-truth migration is formalized at five distinct locations — agent infrastructure, LLM context, cell internals, external systems, and performance optimization stores.

8. Operational test

A deployment exhibits caches-as-authoritative if any of the following are true at any time during the deployment's existence.

1. Cache contains content in one or more of the five source-of-truth categories.
2. Coordination questions are answered from cache content rather than substrate; downstream operations consult cache as authoritative.
3. Cache hits bypass substrate consultation; operations that should consult substrate consult cache instead.
4. Cache staleness produces authoritative-looking answers diverging from substrate; the deployment operates with cache content as authoritative even when divergent.

Three sharpening properties operationalize the test for deployment review.

Cache-content-locus test. Examine cache state for content in the five source-of-truth categories. Presence with operational authority indicates the anti-pattern. The presence of derived content alone does not; what matters is whether downstream operations treat it as the answer to coordination questions.

Cache-vs-substrate-staleness test. Simulate substrate changes without cache invalidation. If operations return stale authoritative content rather than failing or falling back to substrate, the anti-pattern is exhibited.

Cache-miss-substrate-hit test. Examine cache miss handling. If cache misses do not hit substrate (instead falling through to other caches, fallback stores, or default values), the anti-pattern is exhibited even when cache hits behave correctly.

A deployment that satisfies any of (1)–(4) and any of the three sharpening properties exhibits the anti-pattern fully. The architectural correction in §5 specifies the operational changes required.

9. The one-sentence test

If a deployment treats performance caches — Redis, Memcached, in-memory caches, CDN caches, application-level caches, query-result caches, embedding caches, computed-value caches, ORM-level caches, HTTP response caches — as authoritative for coordination questions, with cache hits bypassing substrate consultation and cache staleness producing authoritative-looking answers diverging from substrate, the deployment exhibits caches-as-authoritative; the architectural commitments to substrate-as-source-of-truth and hybrid systems composition both fail through the same mechanism, with Pattern B's non-authoritativeness component directly violated, the composition anti-patterns implicated, read determinism compromised by cache staleness, and the five source-of-truth categories potentially each violated.

10. Why naming the anti-pattern as standalone matters

Implementations under pressure to deliver AI products with strong performance characteristics consistently default to caches-as-authoritative because caching is universally considered good engineering and the language of "performance optimization" makes the architectural failure look like operational success. Audiences understand "we cache for performance" as standard engineering without recognizing the architectural consequence — that caches operating as authoritative violate Pattern B's non-authoritativeness and the foundational substrate-as-source-of-truth commitment fails. Naming the failure mode as standalone — with the four operational components in §2, the violations in §3, the failure mode in §4, the architectural correction in §5, the adjacent-pattern distinctions in §6, the load-bearing argument in §7, and the operational tests in §§8–9 — gives downstream readers a precise specification of the failure and its correction, and supplies prior-art coverage at the location where AI deployments most commonly drift toward source-of-truth migration without operator awareness.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Anti-Pattern: Caches Treated as Authoritative — Standalone Formalization of the Failure Mode Where Performance Optimization Stores Become Operationally Authoritative for Coordination Questions, Violating Substrate-as-Source-of-Truth and Pattern B's Non-Authoritativeness in the Coordination Knowledge Substrate Pattern.* May 6, 2026. ORCID: 0009-0004-8065-3235.